Laboratório Avaliativo

Caio Castro Miranda e Jaime da Cruz Silva

Etapa 3 — Integração REST

1. O que aconteceu com o estoque?

A quantidade disponível foi reduzida normalmente no estoque-db, porque a reserva acontece no Estoque Service antes da criação do pedido. A reserva é uma transação local do Estoque e termina antes de o Pedido persistir qualquer informação, então o estoque pode ficar com menos unidades mesmo que o pedido não seja criado.

2. O pedido foi criado?

Não. Considerando a falha simulada depois da reserva e antes da criação, o pedido não chega a ser salvo no pedido-db e, consequentemente, também não é publicado nenhum evento pedido.criado.

3. Existe uma transação única envolvendo os dois serviços?

Não. Os serviços possuem processos e bancos de dados diferentes, então existem duas transações locais e não uma única transação envolvendo os dois. O @Transactional do Pedido Service, por exemplo, não consegue incluir a operação realizada no banco do Estoque. A chamada REST já foi confirmada no Estoque quando o Pedido ainda pode falhar.

4. Como o sistema poderia desfazer a reserva realizada?

A reserva poderia ser desfeita por meio de uma operação de compensação, devolvendo ao produto a quantidade que havia sido reservada. Essa operação precisa ser idempotente para evitar que uma mesma reserva seja liberada mais de uma vez. Na implementação, isso pode ser feito usando o mesmo correlationId da reserva para identificar a operação.

5. Que mecanismo poderia ser utilizado para realizar essa compensação?

Um mecanismo possível seria uma Saga baseada em eventos, em que cada etapa do processo possui uma ação de compensação quando alguma etapa posterior falha. Nesse caso, o evento estoque.liberar pode ser utilizado para devolver o estoque, principalmente quando o pagamento for rejeitado.

Essa compensação resolve o caso de pagamento rejeitado, mas existe outra situação, que é a falha entre reservar o estoque e criar o pedido. Para tratar essa janela de forma automática seria necessário algum mecanismo adicional, como timeout ou outra estratégia de compensação.

Etapa 4 — Docker Compose

Pedido consegue acessar Estoque?

Sim. O Pedido Service consegue acessar o Estoque por meio da variável ESTOQUE_SERVICE_URL, configurada como http://estoque-service:8080 no Docker Compose. Dessa forma, o Pedido não acessa o banco do Estoque diretamente, mas chama a API do Estoque para fazer a reserva.

Etapa 5 — Introduzindo RabbitMQ

1. Por que o Pedido Service publica em um Exchange em vez de enviar diretamente para uma Queue?

Porque o Pedido Service não precisa conhecer diretamente quem vai consumir a mensagem. Ele publica no Exchange e informa a routing key, e o RabbitMQ fica responsável por encaminhar a mensagem para a fila correspondente.

Isso deixa a comunicação mais desacoplada, porque podemos adicionar outros consumidores ou outras filas sem precisar alterar o serviço que publicou o evento. No laboratório, o Pedido publica no pedidos.exchange usando a routing key pedido.criado.

2. Qual é a diferença entre Exchange, Queue e Consumer?

O Exchange recebe a mensagem publicada e decide para onde ela deve ser encaminhada de acordo com as regras de roteamento. A Queue é onde a mensagem fica armazenada enquanto aguarda o processamento. Já o Consumer é o serviço ou processo que recebe a mensagem da fila e executa o processamento.

Neste caso, o Pedido Service publica no pedidos.exchange, a mensagem é encaminhada para a fila pedido.criado e o Pagamento Service atua como consumidor dessa fila.

3. O Pedido Service sabe quem consumirá o evento?

Não. O Pedido Service conhece o Exchange e a routing key utilizados para publicar a mensagem, mas não precisa saber qual serviço ou qual instância vai consumir o evento.

Isso permite, por exemplo, que o Pagamento Service tenha mais de uma instância consumindo a mesma fila ou que fique temporariamente indisponível sem que o Pedido precise ficar aguardando ou seja alterado por causa disso.

Etapa 8 — Simulação de Falha

1. O pedido foi criado?

Sim. O Pedido Service consegue criar o pedido mesmo com o Pagamento Service parado, porque o processamento do pagamento acontece de forma assíncrona. O pedido é criado inicialmente com o status AGUARDANDO_PAGAMENTO.

2. O estoque foi atualizado?

Sim. A reserva do estoque acontece antes da criação do pedido e antes da publicação da mensagem no RabbitMQ. Portanto, mesmo com o Pagamento Service indisponível, a quantidade do produto é reduzida no Estoque.

3. O sistema inteiro parou?

Não. Apenas o processamento do pagamento ficou temporariamente parado. O Pedido Service, o Estoque Service, os bancos de dados e o RabbitMQ continuam funcionando normalmente.

Isso acontece porque os serviços são independentes e o Pedido não precisa chamar o Pagamento diretamente para concluir a criação do pedido.

4. A mensagem foi perdida? Apresente evidências obtidas a partir da fila do RabbitMQ e dos logs.

Não. A mensagem não foi perdida porque a fila pedido.criado continua armazenando a mensagem enquanto não existe um consumidor disponível.

A evidência disso pode ser observada no RabbitMQ Management, onde a fila apresenta mensagens em Ready e nenhum consumidor conectado enquanto o Pagamento Service está parado. Nos logs do Pedido também é possível verificar que o pedido foi criado e que o evento pedido.criado foi publicado, enquanto não existe um novo processamento correspondente nos logs do Pagamento.

```
C:\Users\caiom\projects\lab-microsservico-ufg>docker compose stop pagamento-service
[+] stop 1/1
 ✓ Container lab-microsservico-ufg-pagamento-service-1 Stopped                    1.4s

C:\Users\caiom\projects\lab-microsservico-ufg>docker compose ps
NAME                                  IMAGE                                  COMMAND                  SERVICE           CREAT
ED              STATUS                    PORTS
lab-microsservico-ufg-estoque-db-1        postgres:16                            "docker-entrypoint.s…"   estoque-db        About
 a minute ago   Up About a minute (healthy)   5432/tcp
lab-microsservico-ufg-estoque-service-1   lab-microsservico-ufg-estoque-service   "java -jar /app.jar"     estoque-service   About
 a minute ago   Up 55 seconds                 0.0.0.0:8081->8080/tcp, [::]:8081->8080/tcp
lab-microsservico-ufg-pagamento-db-1      postgres:16                            "docker-entrypoint.s…"   pagamento-db      About
 a minute ago   Up About a minute (healthy)   5432/tcp
lab-microsservico-ufg-pedido-db-1         postgres:16                            "docker-entrypoint.s…"   pedido-db         About
 a minute ago   Up About a minute (healthy)   5432/tcp
lab-microsservico-ufg-pedido-service-1    lab-microsservico-ufg-pedido-service    "java -jar /app.jar"     pedido-service    About
 a minute ago   Up 54 seconds                 0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
lab-microsservico-ufg-rabbitmq-1          rabbitmq:3.13-management               "docker-entrypoint.s…"   rabbitmq          About
 a minute ago   Up About a minute (healthy)   0.0.0.0:5672->5672/tcp, [::]:5672->5672/tcp, 0.0.0.0:15672->15672/tcp, [::]:15672->1
5672/tcp

C:\Users\caiom\projects\lab-microsservico-ufg>curl.exe -s http://localhost:8081/produtos/2
{"id":2,"nome":"Mouse","quantidade":50}
C:\Users\caiom\projects\lab-microsservico-ufg>curl.exe -s -w "\nHTTP %{http_code}\n" -X POST http://localhost:8080/pedidos -H "Con
tent-Type: application/json" -d "{\"produtoId\":2,\"quantidade\":1}"
{"id":1,"produtoId":2,"quantidade":1,"status":"AGUARDANDO_PAGAMENTO","correlationId":"5368678b-e044-42e7-b616-a4d21483e0f0"}
HTTP 201
```

OBS.: Evidências a frente são da última entrega de evidências (final do .pdf da atividade), mas também suportam aqui.

Get messages

Warning: getting messages from a queue is a destructive action. ?

Ack Mode: Nack message requeue true

Encoding: Auto string / base64 ?

Messages: 1

Get Message(s)

Message 1

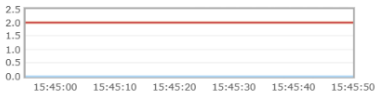
The server reported 1 messages remaining.

Exchange	pedidos.exchange
Routing Key	pedido.criado
Redelivered	○
Properties	priority: 0 delivery_mode: 2 headers: __TypeId__: br.edu.microsservicos.eventos.PedidoCriadoEvento content_encoding: UTF-8 content_type: application/json
Payload 98 bytes Encoding: string	{"pedidoId":1,"produtoId":2,"quantidade":1,"correlationId":"f8b22cb9-a827-45e5-877e-744e60c817ed"}

Queue pedido.criado

Overview

Queued messages last minute ?



Ready	2
Unacked	0
Total	2

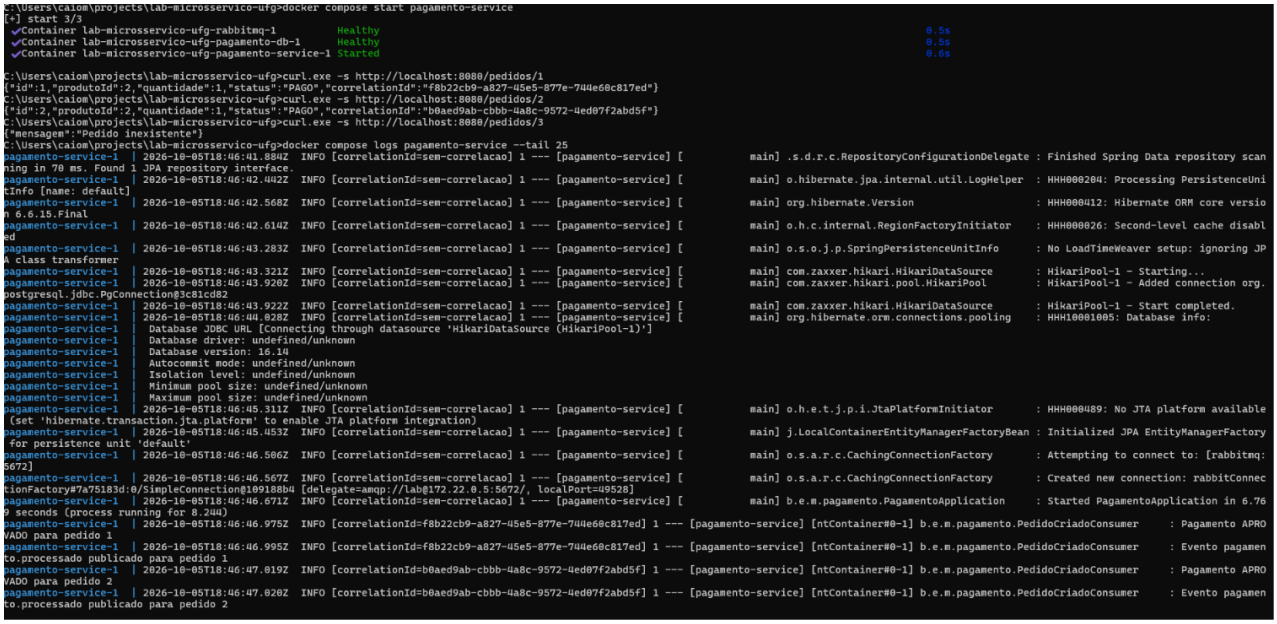
Message rates last minute ?



Publish	0.00/s	Consumer ack	0.00/s	Get (auto ack)	0.00/s
Deliver (manual ack)	0.00/s	Redelivered	0.00/s	Get (empty)	0.00/s
Deliver (auto ack)	0.00/s	Get (manual ack)	0.00/s		

Details

Features	durable: true	State	idle		Total	Ready	Unacked	In memory	Persistent	Transient, Paged Out
	queue storage version: 1	Consumers	0	Messages ?	2	2	0	2	2	0
Policy		Consumer capacity ?	0%	Message body bytes ?	196 B	196 B	0 B	196 B	196 B	0 B
Operator policy				Process memory ?	13 KiB					
Effective policy definition										



(Evidências também estão na pasta evidências)

Etapa 9 — Recuperação

1. O processamento precisou ser repetido manualmente?

Não. Depois de executar docker compose start pagamento-service, o serviço volta a consumir as mensagens que estavam aguardando na fila. Não é necessário criar novamente os pedidos nem publicar as mensagens manualmente.

2. O Pedido Service precisou aguardar o Pagamento Service?

Não. O Pedido Service não precisou ficar esperando o Pagamento para retornar a resposta da criação do pedido. O pedido foi criado e ficou com status AGUARDANDO_PAGAMENTO, enquanto o processamento do pagamento aconteceu posteriormente de forma assíncrona.

3. O que aconteceu com as mensagens enquanto o consumidor estava indisponível?

As mensagens permaneceram armazenadas na fila pedido.criado enquanto o Pagamento Service estava parado. Quando o serviço foi iniciado novamente, ele passou a consumir as mensagens pendentes e processou os pagamentos correspondentes.

Etapa 10 — Escalabilidade

As mensagens foram distribuídas?

Sim. Quando o Pagamento Service é executado com duas instâncias, as duas ficam conectadas à mesma fila pedido.criado. O RabbitMQ distribui as mensagens entre os consumidores disponíveis, fazendo com que as instâncias pagamento-service-1 e pagamento-service-2 processem pedidos diferentes.

Apenas uma instância processou cada mensagem?

Sim. Uma determinada mensagem da fila é entregue a apenas um dos consumidores disponíveis. Dessa forma, duas instâncias não processam normalmente a mesma mensagem ao mesmo tempo.

Ainda assim, como sistemas assíncronos podem ter reentregas em algumas situações, a aplicação também precisa lidar com possíveis duplicidades. No caso do pagamento, o pedidoId pode ser utilizado para evitar que o mesmo pagamento seja registrado novamente.

Observe o comportamento das duas instâncias de Pagamento. Em seguida, discuta quais características da arquitetura permitem que apenas esse serviço seja escalado independentemente dos demais.

O Pagamento Service pode ser escalado de forma independente porque ele recebe os pedidos por meio de uma fila, em vez de depender de uma chamada REST direta feita pelo Pedido. As instâncias do Pagamento podem consumir a mesma fila e dividir o processamento entre elas.

Além disso, o Pagamento possui seu próprio banco de dados e não precisa acessar o banco do Pedido. Dessa forma, aumentar a quantidade de instâncias do Pagamento não exige aumentar ou modificar os outros serviços.

Em quais circunstâncias o Estoque Service também precisaria ser escalado?

O Estoque precisaria ser escalado caso a quantidade de requisições síncronas aumentasse a ponto de uma única instância não conseguir atender a demanda, por exemplo, com muitos GET /produtos ou várias reservas acontecendo ao mesmo tempo.

Mesmo com várias instâncias, operações sobre o mesmo produto ainda podem sofrer concorrência no banco, principalmente porque a quantidade disponível precisa ser atualizada de forma consistente. Então, a escala ajuda principalmente quando existe um volume maior de requisições e quando elas podem ser distribuídas entre diferentes produtos.

Etapa 11 — Observabilidade — Investigação de Incidente

1. O pedido foi criado?

Isso pode ser confirmado verificando os logs do Pedido Service ou consultando GET /pedidos/17. Se existir o registro de criação do Pedido 17, então essa etapa foi concluída.

Caso não exista o pedido, o problema aconteceu antes da criação, podendo estar relacionado, por exemplo, à falta de estoque, produto inexistente ou indisponibilidade do Estoque Service.

2. O estoque foi reservado?

A reserva pode ser confirmada verificando o log do Estoque Service com o correlationId correspondente ao Pedido 17 e também conferindo se a quantidade do produto foi reduzida.

Como a reserva acontece antes da criação do pedido, é possível inclusive existir uma reserva sem que o pedido tenha sido criado caso ocorra uma falha nesse intervalo.

3. O evento pedido.criado foi publicado?

Isso pode ser verificado pelo log do Pedido Service, procurando o registro de publicação do evento pedido.criado com o pedidoId e o correlationId do Pedido 17.

Também é possível verificar a fila pedido.criado no RabbitMQ para saber se a mensagem está aguardando processamento.

4. O Pagamento Service recebeu o evento?

Isso pode ser identificado pelos logs do Pagamento Service e pelo estado da fila pedido.criado. Se existir um log do Pagamento relacionado ao Pedido 17, significa que o evento chegou ao serviço.

Se a mensagem ainda estiver em Ready e o Pagamento estiver sem consumidores, por exemplo, o evento ainda não foi processado porque o serviço está indisponível.

5. O pagamento foi processado?

O pagamento foi processado se houver um registro correspondente no pagamento-db e um log indicando que ele foi aprovado ou rejeitado.

Se o pedido continuar em AGUARDANDO_PAGAMENTO e não existir um registro de pagamento para ele, o processamento ainda não aconteceu.

6. Em qual etapa ocorreu o problema?

A etapa pode ser identificada usando o correlationId para acompanhar o pedido nos três serviços. A primeira etapa que não possuir evidência nos logs indica onde o fluxo parou.

Por exemplo, se o Pedido registrou a criação e a publicação, mas não existe nenhum registro do Pagamento, o problema provavelmente está no consumo do evento pedido.criado. Se o Pagamento registrou o processamento, mas o Pedido não foi atualizado, o problema provavelmente está no consumo do evento de retorno do pagamento.

7. Qual evidência nos logs permite identificar a etapa da falha?

O principal elemento é o correlationId, porque ele permite acompanhar a mesma operação nos diferentes serviços. A última mensagem registrada com esse identificador mostra até onde o fluxo conseguiu chegar.

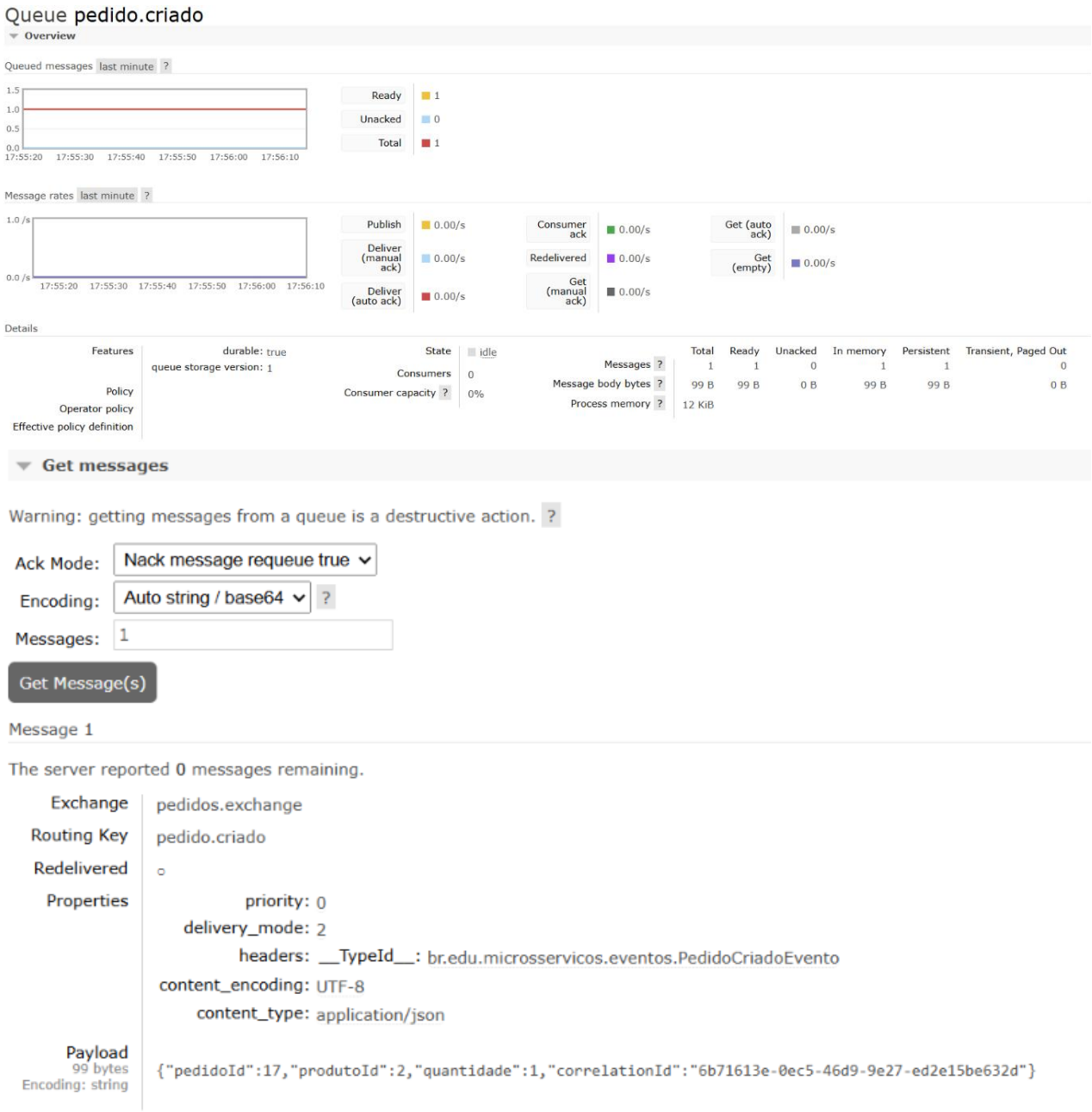
Por exemplo, se aparecem os logs de criação do pedido, reserva do estoque e publicação do evento, mas não aparece nenhum log do Pagamento com o mesmo correlationId, existe uma evidência de que a falha aconteceu antes ou durante o consumo do evento pelo Pagamento.

8. O que aconteceu com a mensagem no RabbitMQ?

Se o Pagamento Service estava indisponível, a mensagem permaneceu na fila pedido.criado, aguardando um consumidor. Isso pode ser observado pela quantidade de mensagens em Ready e pela ausência de consumidores conectados à fila.

Quando o Pagamento Service volta a funcionar, ele se conecta à fila e começa a consumir as mensagens pendentes, permitindo que o processamento continue sem precisar criar novamente o pedido.




Queue pedido.criado
Overview
Queued messages last minute ?
Ready 1
Unacked 0
Total 1
Message rates last minute ?
Publish 0.00/s
Deliver (manual ack) 0.00/s
Deliver (auto ack) 0.00/s
Consumer ack 0.00/s
Redelivered 0.00/s
Get (manual ack) 0.00/s
Get (auto ack) 0.00/s
Get (empty) 0.00/s
Details
Features durable: true
queue storage version: 1
Policy
Operator policy
Effective policy definition
State idle
Consumers 0
Consumer capacity ? 0%
Messages ? Total 1 Ready 1 Unacked 0 In memory 1 Persistent 1 Transient, Paged Out 0
Message body bytes ? 99 B 99 B 0 B 99 B 99 B 0 B
Process memory ? 12 KiB
Get messages
Warning: getting messages from a queue is a destructive action. ?
Ack Mode: Nack message requeue true
Encoding: Auto string / base64 ?
Messages: 1
Get Message(s)
Message 1
The server reported 0 messages remaining.
Exchange pedidos.exchange
Routing Key pedido.criado
Redelivered o
Properties
priority: 0
delivery_mode: 2
headers: __TypeId__: br.edu.microsservicos.eventos.PedidoCriadoEvento
content_encoding: UTF-8
content_type: application/json
Payload
99 bytes
Encoding: string
{"pedidoId":17,"produtoId":2,"quantidade":1,"correlationId":"6b71613e-0ec5-46d9-9e27-ed2e15be632d"}

LOGS:



(ESTÃO NA PASTA EVIDENCIAS TAMBÉM)

Etapa 12 — Atualização assíncrona do pedido

A etapa 12 não apresenta perguntas discursivas no enunciado, mas determina um teste que precisa ser realizado: devem ser criados pedidos suficientes para observar pelo menos um pagamento aprovado e um rejeitado, verificando se o status do pedido é atualizado corretamente nos dois casos.

Nesse fluxo, quando o pagamento é aprovado, o Pagamento Service publica o evento pagamento.processado com status APROVADO, e o Pedido Service recebe esse evento e altera o pedido para PAGO.

Quando o pagamento é rejeitado, o evento é publicado com status REJEITADO, e o Pedido Service altera o pedido para REJEITADO. Em nenhum dos dois casos o Pagamento Service acessa diretamente o banco do Pedido Service, mantendo a comunicação entre os serviços por meio dos eventos.